# /memory

Memory panel (Alt+M): /memory | /memory show | /memory clear | /memory help

## Overview

`/memory` toggles the memory side panel and prints a structured-memory summary into the transcript. The panel view, the transcript summary, and `/memory clear` are the implemented behaviours: the bare form and `/memory show` do the same thing, `/memory clear` opens a `Yes`/`No` confirmation dialog that empties this project's memories, and `/memory help` prints the usage line.

The command description in the slash-command registry lists exactly the implemented forms (`/memory`, `/memory show`, `/memory clear`, `/memory help`). Any argument other than these prints the usage line. This documentation records the code truth so readers do not hunt for behaviour that does not exist.

For real memory operations use the `memory_*` tools (`memory_store`, `memory_recall`, `memory_forget`) in chat, or `Alt+M` to toggle the panel.

## Syntax

```
/memory          # toggle panel + transcript summary (same as show)
/memory show     # same as bare form
/memory clear    # ask Yes/No, then clear this project's memories
/memory help     # prints the usage line
```

## Options / Subcommands

| Form | Description |
|---|---|
| `/memory` | Toggle the memory side panel; print the structured-memory summary |
| `/memory show` | Identical to the bare form |
| `/memory clear` | Open a `Yes`/`No` confirmation dialog, then clear this project's memories |
| `/memory help` | Prints `Usage: /memory show \| /memory clear \| /memory help` |

Any other argument - including the older `init`, `read <label>`, and `search <query>` forms that the registry no longer advertises - prints the usage line.

| Form | Status |
|---|---|
| `/memory init` | Not implemented - prints usage |
| `/memory read <label>` | Not implemented - prints usage |
| `/memory search <query>` | Not implemented - prints usage |

## Examples

Toggle the panel and show the summary:

```
/memory
```

Same effect via the explicit form:

```
/memory show
```

Check the usage line:

```
/memory help
```

Clear this project's memories (a `Yes`/`No` dialog appears; pick `Yes` to confirm, or `No`/`Esc` to cancel):

```
/memory clear
```

Attempt a documented-but-unimplemented subcommand; you get the usage line:

```
/memory search permissions
```

Manage memories through tools instead of slash commands:

```
memory_recall(query: "tool repeat guard")
memory_store(content: "...", category: "workflow")
```

Toggle the panel with the keyboard shortcut:

```
Alt+M
```

## Output

Panel behaviour:

- Opens or closes the memory side panel.
- Opening the panel closes the log, profile, tasks, and telemetry panels (side panels are mutually exclusive).
- The status line reports `memory panel visible` or `memory panel hidden`.

Transcript summary (printed alongside the panel toggle):

- `**Structured memories:** N` - the count of memories stored for this project directory.
- Up to 50 memory rows grouped by category, each group headed `### <category> (N entries)`.
- Each row: `- **#id** \confidence‘ preview‘` with the preview truncated to 120 characters.

Clear behaviour (`/memory clear`):

- Opens the `Clear this project's memory?` dialog with `Yes` / `No`, `No` selected by default so a stray `Enter` cannot clear memory.
- `Left`/`Right` (or `Tab`) move the selection; `Enter` selects; `Esc` cancels.
- On `Yes` every memory whose project is the current working directory (full path or directory base-name) is deleted and the transcript reports `Cleared N memory entries for this project.` Memories belonging to other projects are never touched.
- `No`/`Esc` leave memory unchanged.

Argument forms with no implementation print exactly:

```
Usage: /memory show | /memory clear | /memory help
```

## Related

- `memory_store` / `memory_recall` / `memory_forget` - the actual memory tools
- `Alt+M` - keyboard shortcut for the same panel toggle
- `/reload` - refresh configuration and skills after editing memory config
- Structured memory store (SQLite) configured under `memory` in `ragent.json`